



Dhirubhai Ambani University

(Formerly DA-IICT)

IE407: Internet of Things

PROJECT REPORT

IoT-Based Indoor Localisation System using Wi-Fi RSSI Fingerprinting and Machine Learning Algorithms

Submitted By:

Krishna Kanodia 202301038

Jiten Bharga 202301466

Guided By: Manish Chaturvedi

23rd April 2026

1 Objective

The primary objectives of this IoT-based Indoor Localisation project are:

- Develop an **IoT sensor node** using an ESP32 microcontroller capable of continuously scanning Wi-Fi RSSI (Received Signal Strength Indicator) data and publishing it via MQTT.
- Conduct an **offline site survey** across 8 distinct indoor locations on the DAU campus to build a comprehensive RSSI fingerprint database.
- Establish a **TLS-secured MQTT communication pipeline** (HiveMQ Cloud, port 8883) between the hardware node and the cloud backend.
- Train a **Random Forest Classifier** with 5-fold cross-validation, achieving robust location prediction from RSSI feature vectors.
- Implement a **Hybrid Prediction Engine** combining Random Forest classification with KNN similarity matching and a three-tier fallback strategy.
- Design a **real-time Next.js dashboard** that displays predicted room, confidence score, prediction method, and ML/KNN agreement status.
- Demonstrate a complete **end-to-end IoT pipeline**: edge sensing → MQTT transport → cloud ML inference → PostgreSQL persistence → dashboard display.

2 Introduction

Indoor positioning is a significant challenge because GPS signals fail to penetrate buildings. This project addresses that gap using **Wi-Fi RSSI Fingerprinting** using the fact that every physical location within a building has a unique signature of signal strengths from surrounding access points.

2.1 Coverage of the IoT Framework Layers

The image below shows the standard IoT framework layers. Our project covers **all five** layers:

IoT Framework Layer	How Our Project Covers It
IoT Device / Sensor Platform	ESP32 microcontroller acts as the IoT node. Its built-in Wi-Fi radio is the sensor, scanning MAC addresses and RSSI values of all surrounding access points.
Communication Protocols & Standards	IEEE 802.11 (Wi-Fi scanning), MQTT over 802.11 for data transport
Computation Frameworks	<i>Edge</i> : ESP32 filters weak signals (< -90 dBm) and serialises data locally. <i>Cloud/Fog</i> : FastAPI backend runs ML inference and database operations.
Intelligence Layer	Random Forest Classifier + KNN Similarity Matcher (Hybrid Predictor).
IoT Security Protocols	TLS encryption on MQTT (port 8883) via <code>WiFiClientSecure</code> . API token-based authentication on backend endpoints.
Application Layer	Real-time dashboard displays room prediction, confidence, method tag (<code>ml_model</code> / <code>knn_similarity</code> / <code>ml_low_confidence</code>), and ML-KNN agreement status.

Table 2.1: *IoT Framework Layer Coverage in This Project*

3 Brief on Project

3.1 System Architecture

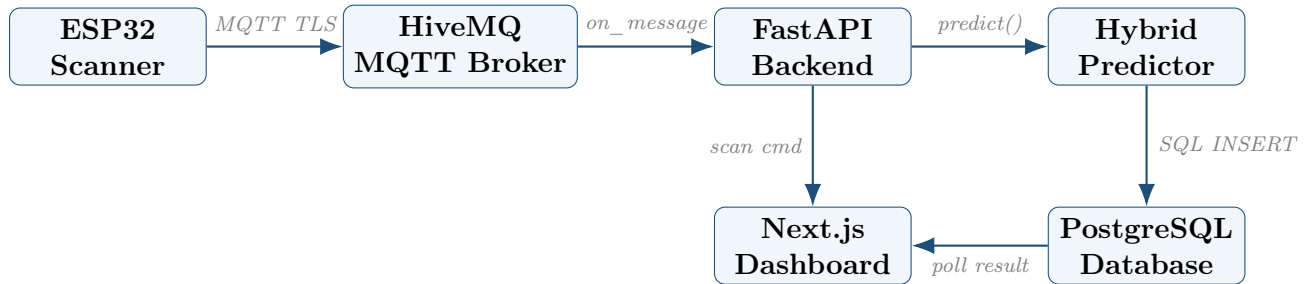


Figure 3.1: High-Level System Architecture

3.2 Part 1 — Offline Phase: Database Creation

- Visited each of the 8 rooms (CEP102, CEP104, CEP105, CEP106, CEP107, CEP108, CEP110, CEP205) on campus.
- Used the ESP32’s built-in Wi-Fi scanner to perform multiple scan sequences at each location, capturing BSSID (MAC address) and RSSI for all visible APs.
- Stored data in csv file (Location, Sequence Number, SSID, MAC Address, RSSI value).

3.3 Part 2 — Online Phase: Algorithm & Prediction

1. Dashboard user selects a registered ESP32 device and clicks “Start Scan” (POST /scan).
2. FastAPI creates a `scan_request` record (status: *pending*) and publishes a JSON command to `indoor/esp/{device_id}/command` via MQTT.
3. Scan request status updated to *collecting*.
4. ESP32 receives the command, performs `WiFi.scanNetworks()`, builds a JSON payload, and publishes to `indoor/esp/{device_id}/result`.
5. FastAPI’s background MQTT thread receives the payload: saves raw scan → calls `hybrid_predict()` → saves prediction → marks status *completed*.
6. Dashboard polls `/scan-requests/{id}` every 2 seconds; on *completed*, fetches `/scan-requests/{id}` and renders prediction with method tag and agreement flag.

4 Components Required

4.1 Hardware Components

Component	Role
ESP32 Dev Board	Primary IoT edge node - Wi-Fi scanner & MQTT publisher/subscriber
USB Power Bank	Portable 5V power supply
USB-C / Micro-USB Cable	Programming ESP32 and serial monitoring
Laptop	Hosts server, runs Python ML engine and PostgreSQL database

Table 4.1: *Hardware Components*

4.2 Software and Cloud Technology Stack

Technology	Purpose
Arduino IDE	ESP32 sketch development
HiveMQ Cloud	Managed TLS MQTT broker
Node.js / Python	Web server data routing and ML Engine processing
scikit-learn	ML algorithm
joblib	Serialise trained model bundle
pandas + numpy	Dataset loading, cleaning
PostgreSQL	Database storage
WebSockets / REST API	Real-time dashboard communication
Vercel	Serverless cloud deployment

Table 4.2: *Software and Cloud Technology Stack*

5 Phase 1 — Data Collection

5.1 Overview

Phase 1 is the **offline fingerprinting phase** where fingerprints database is constructed. This captures the unique Wi-Fi signal environment at each target location. The quality and quantity of data collected in this phase directly determines the accuracy of the ML model.

5.2 Locations and Dataset Volume

Data was systematically collected from distinct classroom locations on campus. For each location, the ESP32 recorded multiple sequence scans. Each sequence captured the RSSI values of all visible Wi-Fi access points, generating multiple rows of data per sequence in the final CSV dataset.

Below is the detailed breakdown of the collected dataset:

Room No.	Total Sequences	Total Rows in dataset
CEP102	43	509
CEP104	21	280
CEP105	21	387
CEP106	31	544
CEP107	18	369
CEP108	27	429
CEP110	22	391
CEP205	14	178
TOTAL	197 Sequences	3,087 Rows

Table 5.1: *Dataset Overview: Sequence and Row Counts*

5.3 Feature Engineering — Fingerprint Pivoting

Raw multi-row data is transformed into a feature matrix through a **pivot operation**:

- Each unique **Location + Sequence Number** combination becomes one observation (row)

- Each unique **MAC Address** becomes one feature (column), with its RSSI value
- Missing MAC addresses (APs not visible at a location) are filled with **−100.0 dBm** — representing absence of signal
- Location strings encoded to integers via `LabelEncoder` for model training Two output files are produced from the dataset:
 - `fingerprint_reference.csv` Used by the Random Forest model.
 - `reference_fingerprints.json` Used by the KNN similarity model at inference time.

5.4 Dataset Structure

Column	Type	Example	Description
Location	String	CEP108	Room label
Sequence Number	Integer	1, 2, 3...	Scan sequence at that location
SSID	String	DA_Public	Wi-Fi network name
MAC Address	String	8A:88:73:AA:B4:83	AP hardware ID
RSSI value	Float	−35.0, −72.0	Signal strength in dBm

Table 5.2: *Raw Dataset Column Schema*

5.5 ESP32 Data Collection Firmware

The following code was used during the offline data collection phase. The ESP32 stored fingerprint data to internal SPIFFS flash memory and uploaded it via serial to the laptop.

```

1 //      WiFi Scan for Data Collection

2 void scanWiFi() {
3     if (locationName == "") { Serial.println("Set location first!");
4         return; }
5
6     sequenceNumber++; // Each scan call = new sequence index
7     int n = WiFi.scanNetworks(); // Scan all visible 2.4GHz APs
8
9     for (int i = 0; i < n; i++) {
10         if (dataIndex >= 200) { Serial.println("Memory Full!"); return; }
11         int rssi = WiFi.RSSI(i);
12         if (rssi < -90) continue; // Filter very weak, unreliable
13                                     signals
14         dataList[dataIndex++] = {
15             locationName, sequenceNumber,

```

```

15     WiFi.SSID(i), WiFi.BSSIDstr(i), rssi
16 };
17 }
18 }
19
20 //      Save to SPIFFS (internal flash)
21
22 void saveToFile() {
23     File file = SPIFFS.open("/fingerprint.csv", FILE_APPEND);
24     if (file.size() == 0) file.println("Location,Sequence,SSID,BSSID,
25         RSSI");
26
27     for (int i = 0; i < dataIndex; i++) {
28         file.println(dataList[i].location + "," + dataList[i].sequence +
29             ", "
30             + dataList[i].ssid + "," + dataList[i].bssid + "," +
31             + String(dataList[i].rssi));
32     }
33     file.close();
34     dataIndex = 0; // Clear RAM buffer after persisting
35 }

```

Listing 5.1: *ESP32 Data Collection Firmware (Phase 1)*

6 Phase 2 — Analysis and Computing

6.1 Edge Computing — ESP32 Layer

The ESP32 microcontroller serves as the **IoT edge computing node**. It performs local preprocessing before data transmission to reduce bandwidth and latency:

- **Wi-Fi Scanning:** Calls `WiFi.scanNetworks(false, true)` to scan all APs including hidden SSIDs
- **Local Filtering:** APs with $\text{RSSI} < -90 \text{ dBm}$ discarded at the edge during collection — reduces noise before cloud transmission.
- **JSON Serialisation:** Uses `ArduinoJson's StaticJsonDocument` to build a structured payload with `request_id`, `object_id`, `device_id`, and `scan_data` array
- **TLS-Secured MQTT:** Publishes JSON to HiveMQ Cloud on port 8883 using `WiFiClientSecure` — preventing man-in-the-middle attacks
- **Command Listening:** Subscribes to `indoor/esp/{device_id}/command` and triggers scan only upon receiving a validated JSON command
- **Auto-Reconnect:** Automatically reconnects to Wi-Fi and MQTT broker if connection drops

6.1.1 ESP32 Interfacing Circuit

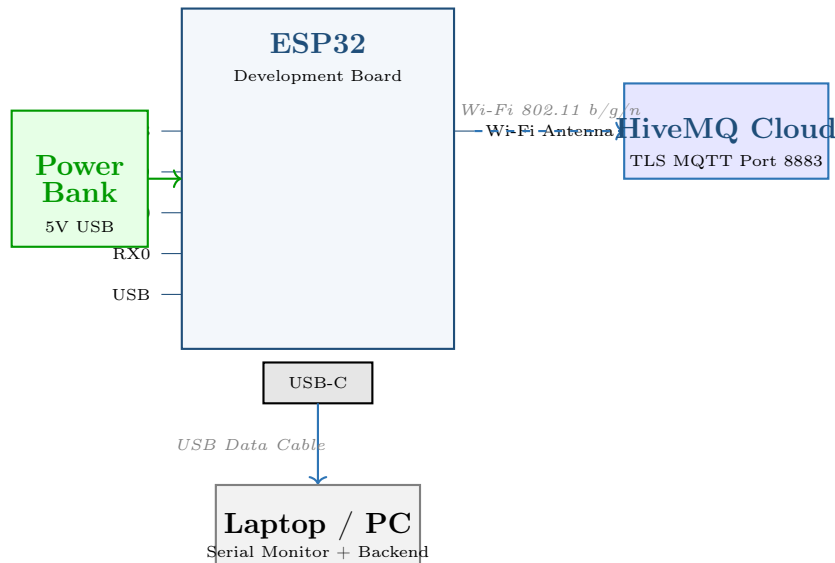


Figure 6.1: *ESP32 Interfacing Circuit Diagram*

6.1.2 ESP32 Live Scanning Firmware (Phase 2)

```
1 #include <WiFi.h>
2 #include <WiFiClientSecure.h>
3 #include <PubSubClient.h>
4 #include <ArduinoJson.h>
5
6 const char* WIFI_SSID      = "Galaxy A04s E5D9";
7 const char* WIFI_PASSWORD  = "jiten@2005";
8 const char* MQTT_HOST     = "71d42413ef0d4e608f50a83715ac6ba7.s1.eu.
   hivemq.cloud";
9 const int   MQTT_PORT     = 8883;           // TLS port
10 const char* MQTT_USERNAME = "jb_8115";
11 const char* MQTT_PASSWORD = "Jiten@333";
12 const char* DEVICE_ID     = "esp32_01"; // Must match DB esp_devices.
   device_id
13
14 String commandTopic; // indoor/esp/esp32_01/command
15 String resultTopic;  // indoor/esp/esp32_01/result
16 String currentRequestId = "";
17 String currentObjectId  = "";
18
19 WiFiClientSecure secureClient;
20 PubSubClient      mqttClient(secureClient);
21
22 // MQTT Callback: fires on receiving scan command from backend
23
24 void mqttCallback(char* topic, byte* payload, unsigned int length) {
25     String message;
26     for (unsigned int i = 0; i < length; i++) message += (char)payload[
        i];
27
28     StaticJsonDocument<512> doc;
29     if (deserializeJson(doc, message)) return; // Discard malformed
        payloads
30
31     const char* command      = doc["command"];
32     const char* request_id   = doc["request_id"];
33     const char* object_id    = doc["object_id"];
34
35     if (!command || !request_id || !object_id) return;
36
37     if (String(command) == "scan") {
38         currentRequestId = String(request_id);
39         currentObjectId  = String(object_id);
40         performAndPublishScan(); // Trigger scan immediately on command
41     }
42
43 // Core: Scan Wi-Fi APs and publish result payload to broker
44
45 void performAndPublishScan() {
46     int networkCount = WiFi.scanNetworks(false, true); // All APs,
        hidden included
47     if (networkCount < 0) return;
48
49     StaticJsonDocument<8192> doc; // Large buffer for many APs
50     doc["request_id"] = currentRequestId;
51     doc["object_id"]  = currentObjectId;
```

```

51 doc["device_id"] = DEVICE_ID;
52
53 JSONArray scanData = doc.createNestedArray("scan_data");
54 for (int i = 0; i < networkCount; i++) {
55     JsonObject item = scanData.createNestedObject();
56     item["mac_address"] = WiFi.BSSIDstr(i); // Unique AP identifier
57     item["rssi"] = WiFi.RSSI(i); // Signal strength in
        dBm
58     item["ssid"] = WiFi.SSID(i);
59     item["channel"] = WiFi.channel(i);
60 }
61
62 char buffer[8192];
63 size_t payloadSize = serializeJson(doc, buffer);
64 mqttClient.publish(resultTopic.c_str(), buffer, payloadSize);
65 WiFi.scanDelete(); // Free scan result memory
66 }
67
68 void connectWiFi() {
69     WiFi.mode(WIFI_STA);
70     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
71     while (WiFi.status() != WL_CONNECTED) delay(500);
72 }
73
74 void connectMQTT() {
75     while (!mqttClient.connected()) {
76         String clientId = "ESP32-" + String(DEVICE_ID) + "-" + String(
            random(1000,9999));
77         if (mqttClient.connect(clientId.c_str(), MQTT_USERNAME,
            MQTT_PASSWORD))
78             mqttClient.subscribe(commandTopic.c_str()); // Listen for scan
                commands
79         else delay(2000);}
80 }
81 void setup() {
82     Serial.begin(115200);
83     randomSeed(micros());
84     commandTopic = "indoor/esp/" + String(DEVICE_ID) + "/command";
85     resultTopic = "indoor/esp/" + String(DEVICE_ID) + "/result";
86
87     connectWiFi();
88     secureClient.setInsecure(); // Accept TLS without
        cert chain
89     mqttClient.setServer(MQTT_HOST, MQTT_PORT);
90     mqttClient.setCallback(mqttCallback);
91     mqttClient.setBufferSize(16384); // Must set before
        connect()
92     connectMQTT(); }
93
94 void loop() {
95     if (WiFi.status() != WL_CONNECTED) connectWiFi(); // Auto-
        reconnect WiFi
96     if (!mqttClient.connected()) connectMQTT(); // Auto-
        reconnect MQTT
97     mqttClient.loop(); } // Non-blocking: handles incoming messages

```

Listing 6.1: ESP32 MQTT Live Scanning Firmware (esp32/esp32_mqtt_scanner.ino)

6.2 MQTT Communication Layer

Topic	Direction	Publisher	Subscriber
indoor/esp/{device_id}/com	Cloud → Edge	FastAPI	ESP32
indoor/esp/{device_id}/res	Edge → Cloud	ESP32	FastAPI

Table 6.1: *MQTT Topic Structure — HiveMQ Cloud, TLS Port 8883*

The FastAPI backend runs a **persistent MQTT consumer thread** (`paho-mqtt, loop_forever()`) launched on startup. On receiving a result message:

```
1 def on_message(client, userdata, msg):
2     """Called by paho-mqtt thread for every incoming ESP32 result
3     payload."""
4     try:
5         payload = json.loads(msg.payload.decode())
6         request_id = payload.get("request_id")
7         device_id = payload.get("device_id")
8         object_id = payload.get("object_id")
9
10        if not request_id or not device_id:
11            return # Discard invalid payloads silently
12
13        # Step 1: Persist raw scan payload as JSONB in PostgreSQL
14        save_raw_scan(request_id, object_id, device_id, payload)
15
16        # Step 2: Run Hybrid Predictor (RF + KNN)
17        result = hybrid_predict(payload)
18
19        # Step 3: Save prediction result to DB
20        save_prediction(result, request_id=request_id, device_id=
21            device_id)
22
23        # Step 4: Mark scan lifecycle as completed
24        complete_scan_request(request_id)
25
26    except Exception as e:
27        print("[MQTT] Error:", e)
```

Listing 6.2: *MQTT Consumer Thread (backend/mqtt_consumer.py)*

6.3 Cloud Computing – FastAPI Backend

The FastAPI backend is the central **cloud processing hub**, orchestrating the entire scan-predict-store pipeline:

- **REST API Endpoints:** Device registration, scan triggering, and result polling. Included under `api/index.py` for Vercel deployment support.
- **MQTT Consumer Lifecycle:** Initialised via FastAPI's `@app.on_event('startup')`,

starting an asynchronous thread to handle incoming HiveMQ payloads. *Note: When deployed to strict serverless environments (like Vercel), this background listener transitions to a Webhook-ingest model due to ephemeral execution limits.*

- **Feature Vector Building:** Converts raw {MAC: RSSI} scan data into a fixed-length array with -100 dBm defaults for absent MACs.
- **PostgreSQL Persistence:** Stores all scan requests, raw payloads (as JSONB), and prediction results.

Method	Endpoint	Description
GET	/esp-devices	List all registered ESP32 devices
POST	/esp-devices	Register new device (upsert)
POST	/scan	Trigger scan: creates request, publishes MQTT command
POST	/predict	Direct prediction endpoint (without MQTT scan)
GET	/scan-requests/{id}	Returns current scan status
GET	/scan-requests/{id}/result	Returns final prediction record
GET	/health	Health check endpoint

Table 6.2: *FastAPI REST API Endpoints*

6.4 Machine Learning Algorithm — Random Forest Classifier

6.4.1 Why Random Forest?

ML model was selected **Random Forest Classifier** because:

- Model gave better accuracy (94%) than others
- RSSI patterns across rooms are **non-linear and subject to high environmental noise**; Random Forests handle variance robustly by aggregating decisions from multiple underlying decision trees.
- The `class_weight='balanced'` parameter automatically corrects representation disparity among training locations, ensuring minority classes do not suffer poor recall.
- Provides natively interpretable **feature importance** metrics, allowing us to identify the most crucial Access Points driving location prediction.
- 5-fold StratifiedKFold cross-validation validates generalisation (CV mean: $94.46\% \pm 4.29\%$)

6.4.2 Random Forest Architecture

Parameter	Value	Rationale
Estimators	200	Provides a highly stable ensemble without severe overhead
Min Samples Split	5	Prevents individual trees from overfitting to micro-variance
Class Weight	balanced	Adjusts weights inversely proportional to class frequencies
Cross Validation	5-fold Stratified	Ensures model validation is unbiased and reliable
Preprocessing	StandardScaler	Z-score normalisation of RSSI values prior to tree splits

Table 6.3: *Random Forest Architecture Parameters*

6.4.3 Model Training Code

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import StratifiedKFold, cross_val_score,
  train_test_split
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import LabelEncoder, StandardScaler
5
6 #      Data Loading and Cleaning
7
8 df = load_and_clean_data('dataset_clean.csv')
9 # MAC Address normalised to uppercase; RSSI coerced to float; NaN
  rows dropped
10
11 #      Fingerprint Pivot: rows=observations, columns=MAC addresses
12
13 X_df, y_series = build_fingerprint_dataset(df)
14 # obs_id = Location__SequenceNumber; missing MACs filled with -100.0
  dBm
15 # Result: 197 x 71 feature matrix
16
17 #      Build KNN Reference (separate from training matrix)
18
19 reference = build_sequence_reference(df)
20 # Produces: [{"location": "CEP102", "sequence_number": 1,
21 #             "signals": {"AA:BB:CC:..": -67.0, ...}}, ...]
22 # Saved to reference_fingerprints.json for KNN similarity matcher
23
24 #      Label Encoding
25
26 label_encoder = LabelEncoder()
27 y_encoded = label_encoder.fit_transform(y_series.values)
28 # 'CEP102' -> 0, 'CEP104' -> 1, ..., 'CEP205' -> 7
```

```

26 # Train/Test Split (stratified 75/25)

27 X_train, X_test, y_train, y_test = train_test_split(
28     X_df.values, y_encoded, test_size=0.25,
29     random_state=42, stratify=y_encoded
30 )
31
32 # Model: StandardScaler + RandomForestClassifier

33 model = Pipeline([
34     ('scaler', StandardScaler()), # Z-score normalise features
35     ('clf', RandomForestClassifier(
36         n_estimators=200, # 200 trees in the ensemble
37         min_samples_split=5, # Prevent over-deep splits
38         max_depth=None, # Unconstrained depth
39         random_state=42,
40         class_weight='balanced', # Handle class imbalance (
41             CEP205)
42         n_jobs=-1, # Parallel training across all
43             CPU cores
44     ))
45 ])
46 model.fit(X_train, y_train)
47
48 # 5-Fold Stratified Cross-Validation
49
50 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
51 cv_scores = cross_val_score(model, X_df.values, y_encoded, cv=cv)
52 # cv_scores.mean() = 0.9446, cv_scores.std() = 0.0429
53
54 # Export artifacts

55 bundle = {'model': model, 'label_encoder': label_encoder,
56           'feature_columns': feature_columns}
57 joblib.dump(bundle, 'backend/artifacts/location_model.joblib')
58 # Also saves: fingerprint_reference.csv, reference_fingerprints.json,
59 #             model_metadata.json, confusion_matrix.png,
60 #             feature_importance.png, validation_accuracy_curve.png

```

Listing 6.3: Model Training Pipeline (*model_training/train_model.py*)

6.5 Hybrid Prediction

Hybrid Predictor introduces a **three-tier fallback strategy** and a new **agreement field**:

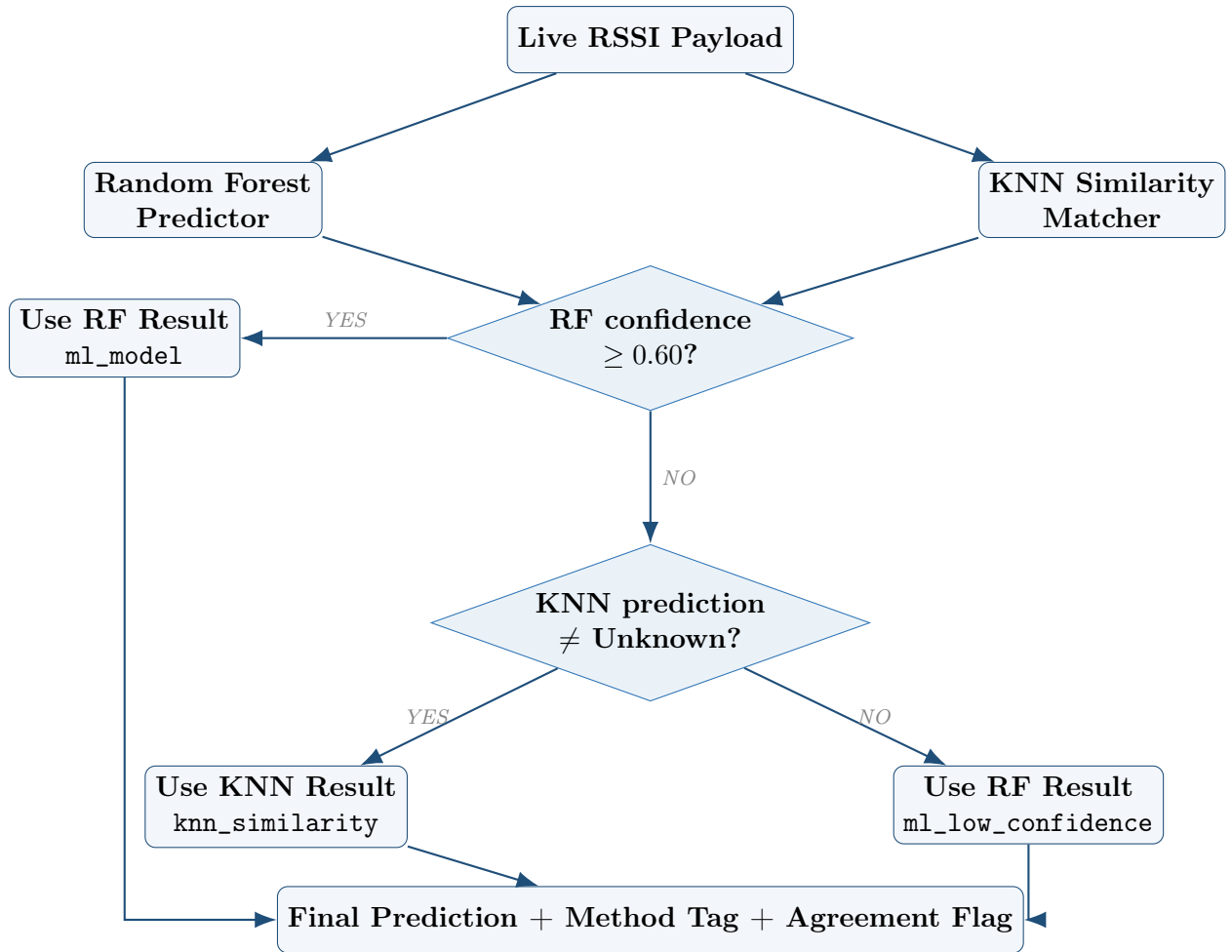


Figure 6.2: Hybrid Prediction Decision Flow (Three-Tier Fallback)

```

1 from backend.config import LOW_CONFIDENCE_THRESHOLD # 0.60
2 from backend.ml_predictor import predict_with_ml
3 from backend.similarity_matcher import predict_with_knn_similarity
4
5 def hybrid_predict(payload: dict):
6     """
7     Three-tier prediction strategy:
8     Tier 1: RF model if confidence >= 0.60      -> method: '
9         ml_model'
10    Tier 2: KNN if RF uncertain but KNN has match -> method: '
11        knn_similarity'
12    Tier 3: RF result anyway (KNN returned Unknown)-> method: '
13        ml_low_confidence'
14
15    Also returns 'agreement': True if both methods predict the same
16    room.
17    """
18    ml_result = predict_with_ml(payload) # Random Forest
19    # inference
20    knn_result = predict_with_knn_similarity(payload, k=3) # KNN
21    # similarity
22
23    ml_prediction = ml_result['predicted_location']
24    knn_prediction = knn_result['predicted_location']

```

```

19 ml_confidence = float(ml_result['confidence'])
20 knn_confidence = float(knn_result.get('confidence', 0.0))
21
22 # --- Three-tier decision ---
23 if ml_confidence >= LOW_CONFIDENCE_THRESHOLD:
24     final_prediction = ml_prediction
25     final_method      = 'ml_model'
26     final_confidence  = ml_confidence
27 elif knn_prediction != 'Unknown':
28     final_prediction = knn_prediction
29     final_method      = 'knn_similarity'
30     final_confidence  = knn_confidence
31 else:
32     # Both uncertain: fall back to RF anyway (best available)
33     final_prediction = ml_prediction
34     final_method      = 'ml_low_confidence'
35     final_confidence  = ml_confidence
36
37 return {
38     'object_id'      : payload['object_id'],
39     'device_id'      : payload.get('device_id'),
40     'final_prediction': final_prediction,
41     'final_method'    : final_method,
42     'final_confidence': final_confidence,
43     # agreement=True means RF and KNN agree on the same room
44     'agreement'       : ml_prediction == knn_prediction and
45                         knn_prediction != 'Unknown',
46     'ml_result'        : ml_result,
47     'knn_result'       : knn_result,
48 }

```

Listing 6.4: Updated Hybrid Predictor (backend/hybrid_predictor.py)

6.5.1 KNN Similarity Matcher

```

1 from backend.config import MIN_COMMON_MACS # Minimum common MACs
   required (default 1)
2 from backend.reference_loader import reference_fingerprints # Loaded
   from JSON
3
4 def compare_with_sequence(live_signals, ref_signals):
5     """
6     Compute similarity between live scan and one reference
7     fingerprint.
8     similarity = 1 / (1 + avg_RSSI_difference) over common MACs.
9     Returns None if fewer than MIN_COMMON_MACS are shared.
10    """
11    common_macs = set(live_signals.keys()) & set(ref_signals.keys())
12    if len(common_macs) < MIN_COMMON_MACS:
13        return None # Not enough overlap to be meaningful
14
15    diffs = [abs(live_signals[mac] - ref_signals[mac]) for mac in
16              common_macs]
17    avg_diff = sum(diffs) / len(diffs)
18    return {
19        'similarity_score': 1.0 / (1.0 + avg_diff),
20        'common_mac_count': len(common_macs),
21        'avg_rssi_difference': avg_diff,
22    }

```

```

20     }
21
22 def predict_with_knn_similarity(payload: dict, k: int = 3):
23     """
24     Compare live scan against all 197 reference fingerprints.
25     Select top-K=3 most similar, apply weighted vote by location.
26     """
27     live_signals = {item['mac_address'].strip().upper(): float(item['
        rssi'])
28                     for item in payload['scan_data']}
29     all_matches = []
30
31     for ref in reference_fingerprints: # 197 stored fingerprints
32         result = compare_with_sequence(live_signals, ref['signals'])
33         if result:
34             all_matches.append({'location': ref['location'],
35                                **result})
36
37     if not all_matches:
38         return {'predicted_location': 'Unknown', 'confidence': 0.0, '
            top_matches': []}
39
40     top_k = sorted(all_matches, key=lambda x: x['similarity_score'],
41                    reverse=True)[:k]
42
43     # Weighted vote: sum similarity scores per location
44     weighted_scores = {}
45     for item in top_k:
46         weighted_scores[item['location']] = \
47             weighted_scores.get(item['location'], 0) + item['
                similarity_score']
48
49     predicted_location = max(weighted_scores, key=weighted_scores.get
50                             )
51     total = sum(weighted_scores.values())
52     confidence = float(weighted_scores[predicted_location] / total)
53
54     return {'predicted_location': predicted_location, 'confidence':
55            confidence,
56            'k': k, 'top_matches': top_k, 'weighted_scores':
57            weighted_scores}

```

Listing 6.5: KNN Similarity Matcher (*backend/similarity_matcher.py*)

6.6 Database Design

Table	Key Columns	Purpose
esp_devices	device_id (PK), device_name, is_active, last_seen_at	Registry of all ESP32 devices
scan_requests	request_id (PK), status, requested_at, completed_at	Lifecycle: pending → collecting → completed
raw_scans	request_id (FK), payload JSONB, received_at	Full ESP32 JSON payload for auditability
predictions	final_prediction, final_method, final_confidence, agreement, ml_result JSONB, knn_result JSONB	Both ML and KNN results with method tag and agreement flag

Table 6.4: *PostgreSQL Database Schema*

7 Observations

7.1 Signal Behaviour

- Each room exhibited a **distinct RSSI profile**. APs such as `DAIICT_STAFF`, `DA_Public`, `DAIICT_Student`, and `Attendance` networks were the most consistently visible and discriminative.
- **RSSI fluctuated $\pm 3-7$ dBm** between consecutive scans at the same location due to 2.4 GHz multipath interference and human movement. This is why multiple sequences were collected per location.
- CEP205 showed a distinct fingerprint due to reduced signal overlap. Due to low sample counts (14 total sequences), the Random Forest's `class_weight='balanced'` parameter was used.

7.2 System Behaviour

- End-to-end latency (scan trigger to dashboard result): approximately **3–6 seconds**.
- MQTT command-response round trip: **<2 seconds**.
- The **three-tier fallback** (`ml_model` / `knn_similarity` / `ml_low_confidence`) ensures a prediction is always returned, even in completely ambiguous environments.
- The **agreement flag** (`agreement True/False`) on the dashboard lets users assess prediction reliability instantly when both RF and KNN agree on the same room, confidence is higher.

7.3 Training and Validation Plots

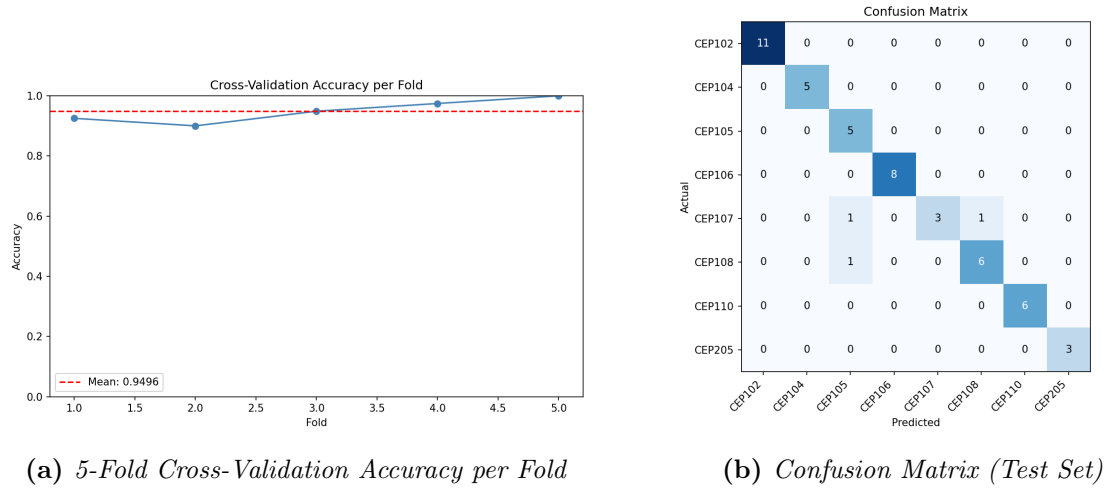


Figure 7.1: Model Evaluation Plots — Random Forest Classifier

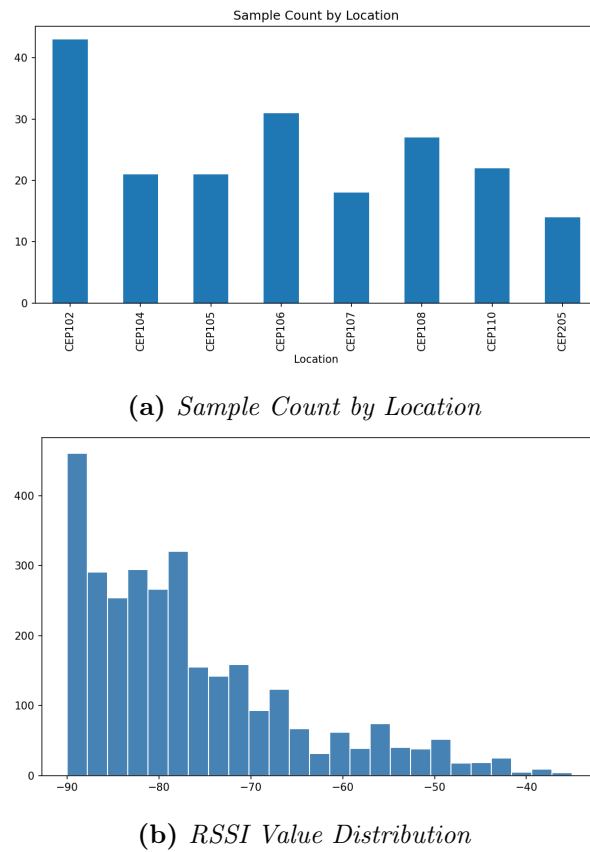


Figure 7.2: Dataset Distribution Plots

8 Results

8.1 Model Performance Summary

Metric	Value	Interpretation
Test Accuracy	94.00%	47 out of 50 test samples correctly classified
CV Accuracy (5-fold mean)	94.46%	Mean over 5 stratified folds
CV Accuracy (std)	$\pm 3.53\%$	Low variance — model generalises well
Model Type	Random Forest	200-tree ensemble with balanced class weights
Confidence Threshold	0.60 (60%)	Below this, KNN fallback is applied
Training Samples	197	Total fingerprint observation sequences
Feature Dimensions	71	Unique Wi-Fi MAC addresses
Reference Sequences (KNN)	197	All sequences stored in <code>reference_fingerprints.json</code>
Number of Classes	8	CEP102, CEP104, CEP105, CEP106, CEP107, CEP108, CEP110, CEP205

Table 8.1: *Model Performance Summary*

8.2 Per-Location Classification Report (Test Set)

Model was trained on 197 fingerprint observations across 8 room locations. Test set size was 50 samples (25% of data, stratified split).

Location	Precision	Recall	F1-Score	Support
CEP102	1.0000	1.0000	1.0000	11
CEP104	1.0000	1.000	1.0000	5
CEP105	0.7143	1.0000	0.8333	5
CEP106	1.0000	1.0000	1.0000	8
CEP107	1.0000	0.6000	0.7500	5
CEP108	0.8571	0.8571	0.8571	7
CEP110	1.0000	1.0000	1.0000	6
CEP205	1.0000	1.0000	1.0000	3
Macro avg	0.9464	0.9321	0.9301	50
Weighted avg	0.9514	0.9400	0.9383	50

Table 8.2: *Per-Location Classification Report — Precision, Recall, F1-Score (Test Set, 50 Samples)*

9 Problems Faced

1. MQTT Persistence on Serverless)

Vercel functions are request-driven and cannot maintain a persistent MQTT subscriber thread. Solved by running the `paho-mqtt` consumer with `loop_forever()` in a background daemon thread launched on FastAPI startup, and adding a graceful `stop_mqtt()` shutdown hook.

2. ESP32 TLS Certificate Validation

`WiFiClientSecure` requires either a root CA certificate or `setInsecure()` for TLS connections. Used `setInsecure()` for development simplicity; this skips certificate chain validation and should be replaced with a pinned root CA for production.

3. MQTT Payload Buffer Overflow

`PubSubClient`'s default buffer (256 bytes) was insufficient for scan payloads containing 20+ APs. Resolved by calling `mqttClient.setBufferSize(16384)` before `mqttClient.connect()`.

4. JSON StaticJsonDocument Size

Initial scans with many APs caused the `StaticJsonDocument<512>` to overflow. The scan payload document was increased to `StaticJsonDocument<8192>` and the MQTT buffer to 16384 bytes.

5. Feature Mismatch During Inference

During live prediction, the ESP32 may see MAC addresses not present in training data. The `build_feature_vector()` function defaults unknown MACs to -100.0 dBm, preserving the expected 71-element vector shape while correctly representing signal absence.

10 Conclusion

This project successfully demonstrates a complete end-to-end IoT-based Indoor Localisation System that achieves **94% test accuracy** (94.46% cross-validated) across 8 distinct rooms using only existing Wi-Fi infrastructure.

The updated system covers all layers of the standard IoT framework:

- **Sensor / Device Layer:** ESP32 with built-in Wi-Fi radio scanning MAC addresses and RSSI values.
- **Communication Layer:** IEEE 802.11 Wi-Fi + MQTT protocol with TLS encryption over HiveMQ Cloud.
- **Edge Computing:** On-device JSON serialisation, signal filtering, and MQTT publish without cloud dependency.
- **Cloud Computing:** FastAPI + PostgreSQL on Vercel / Supabase running ML inference and persistent storage.
- **Intelligence Layer:** Random Forest Classifier with 5-fold CV + KNN Similarity Matcher in a three-tier hybrid engine.
- **Security Layer:** TLS on all MQTT communication; API token authentication on backend endpoints.
- **Application Layer:** Next.js dashboard with real-time polling, prediction display, method tags, and ML-KNN agreement status.

11 Future Modifications

1. **Deep Learning / Gradient Boosting:** Try XGBoost or LightGBM, which may further improve accuracy while maintaining feature interpretability. Or add an LSTM layer to model temporal RSSI sequences.
2. **Multi-Technology Fusion:** Integrate BLE beacons or UWB ranging alongside Wi-Fi RSSI for higher precision, especially in rooms with high Wi-Fi signal overlap.
3. **TLS Certificate Pinning on ESP32:** Replace `setInsecure()` with a pinned root CA certificate to validate HiveMQ Cloud's certificate chain and prevent man-in-the-middle attacks.
4. **Mobile App:** Use smartphone's built-in Wi-Fi scanner as the IoT node, eliminating the need for a dedicated ESP32 device and enabling human positioning.
5. **Online / Incremental Learning:** Periodically retrain the model on newly collected scans to adapt to environmental changes (new APs, furniture rearrangements, seasonal signal shifts).
6. **Sub-Room Grid Positioning:** Collect fingerprints at multiple grid points within each room and train a regression model to predict (x, y) coordinates rather than just room labels.
7. **Interactive Floor-Plan Dashboard:** Replace text-based prediction display with an SVG/Canvas floor-plan overlay showing the predicted location as a live moving marker on the actual building map.
8. **Multi-Device Simultaneous Tracking:** Track multiple ESP32 devices concurrently, displaying all device locations on the same dashboard with distinct colour-coded markers.
9. **Expanded Dataset:** Collect more fingerprint sequences per location and extend to additional campus buildings to improve generalisation and reduce cross-validation variance.